



TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

Scaling neural operators for large-scale plasma simulations and coupling to production codes

October 2026 | Nikola Mamić | JSC

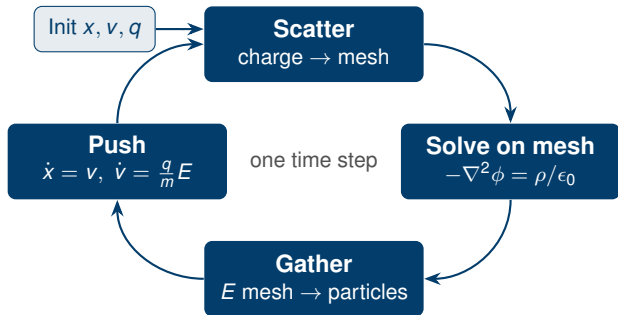


Introduction

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

PARTICLE–FIELD SYSTEMS AND THE PIC LOOP

- **Particles** move by ODEs, **fields** obey PDEs
- Coupled in both directions: particles make the field, the field pushes the particles
- Plasmas (particle-in-cell, PIC), vortex flows, ...
- Electrostatic plasma:
 $-\nabla^2 \phi = \rho / \epsilon_0, \quad E = -\nabla \phi$



FROM GRIDS TO FOURIER MODES

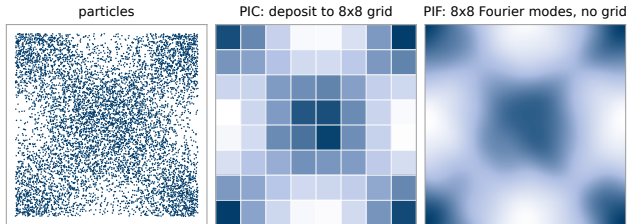
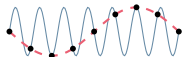


Illustration with synthetic particles, both with 64 degrees of freedom

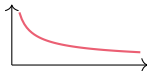
PIC: the grid causes trouble



aliasing → grid
heating, energy drift



cells must resolve
the **Debye length**



noise $\propto N_{\text{ppc}}^{-1/2}$
(particles per cell)

PIF: particles talk to Fourier modes

- $\hat{\rho}_k = S_k \sum_j q_j e^{-ik \cdot x_j}$ — no real-space grid
- No aliasing, spectral accuracy
- With NUFFT: same cost order as PIC, higher constant

WHY SURROGATES?

$\sim 10^{11}$
grid points / modes

$\sim 10^{12}$
particles

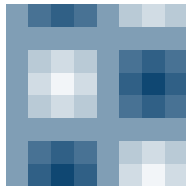
$\sim 10^5$
time steps

for one high-fidelity run (fusion, particle accelerators)

- PIC and PIF **do** scale well on today's supercomputers
 - The problem is **cost**, not scaling
 - Many-query tasks (many runs per study) are out of reach even at exascale
- ⇒ Need **cheap surrogates** that keep the essential physics

DEEP LEARNING FOR PDES SO FAR

- **Foundation models**: pretrain once on many PDEs, fine-tune for new tasks
- **Poseidon** (Herde et al., NeurIPS 2024): pretrained on compressible Euler and incompressible Navier–Stokes data; vision-transformer-based; generalises to unseen PDEs
- But:
 - input = fields on **regular grids** (images), not particles
 - not coupled to production simulation codes

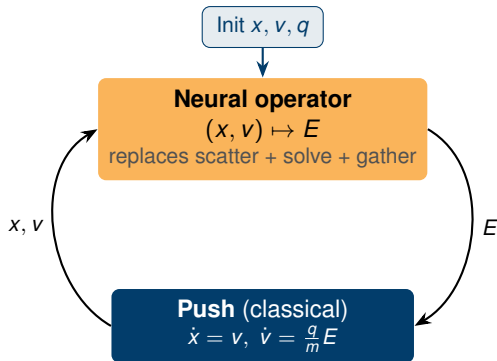


fields on a grid
(Poseidon)



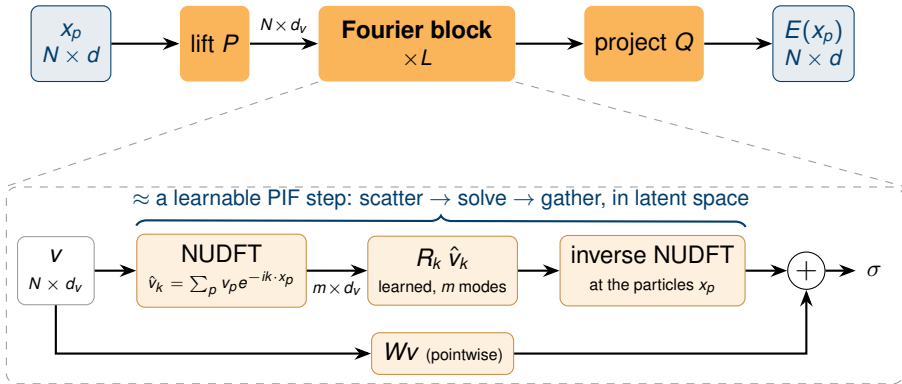
particles anywhere
(our data)

OUR APPROACH: NEOPIC



- Learn the whole map $P^T \mathcal{L}^{-1} P : (x, v) \mapsto E$
- Particle push stays classical
- Training data from mini-apps (PIF/PIC): Landau, two-stream, bump-on-tail, cyclotron; 600 steps, 500k particles each
- Input: particle positions $\rightarrow$ output: E at the particles

THE MODEL: NONUNIFORM FNO



Positions enter through the **Vandermonde matrix** $V_{kp} = e^{-ik \cdot x_p}$: no grid, any particle count

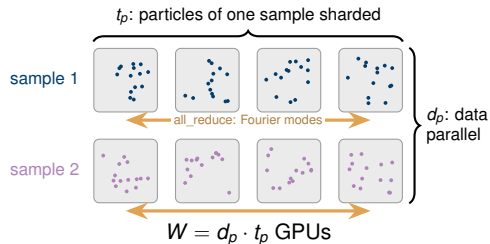


The project

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

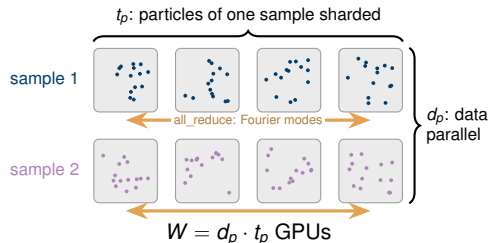
WHERE THE PROJECT STOOD

- ✓ NUFNO model
- ✓ Distributed training: data parallel (DDP)
× particle decomposition
implemented by Chelsea John
- ✓ picND: evaluation against a PIF reference
- ✓ Training data from the mini-apps



WHERE THE PROJECT STOOD

- ✓ NUFNO model
- ✓ Distributed training: data parallel (DDP)
× particle decomposition
implemented by Chelsea John
- ✓ picND: evaluation against a PIF reference
- ✓ Training data from the mini-apps



Next steps — my part:

Scaling inference out

from one GPU holding all particles to many GPUs

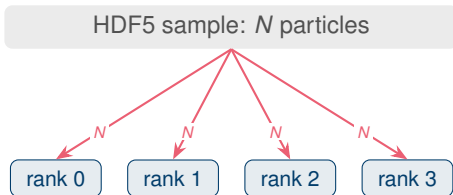
Leaner I/O

each rank reads only the particles it needs

Into a production code

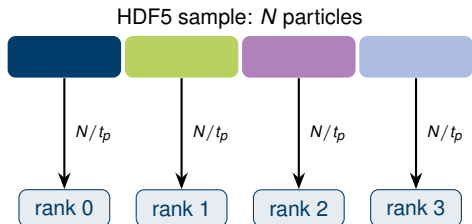
run the model inside a real simulation

TRAINING I/O: SHARDED HDF5 READS



before: each rank reads N , keeps N/t_p

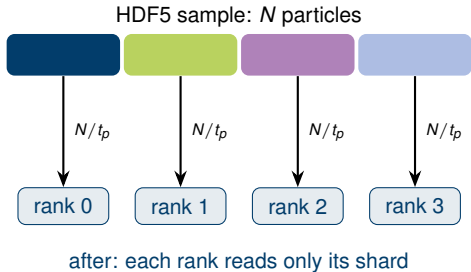
TRAINING I/O: SHARDED HDF5 READS



after: each rank reads only its shard

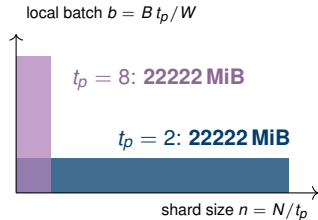
- Read volume per rank: $N \rightarrow N/t_p$

TRAINING I/O: SHARDED HDF5 READS



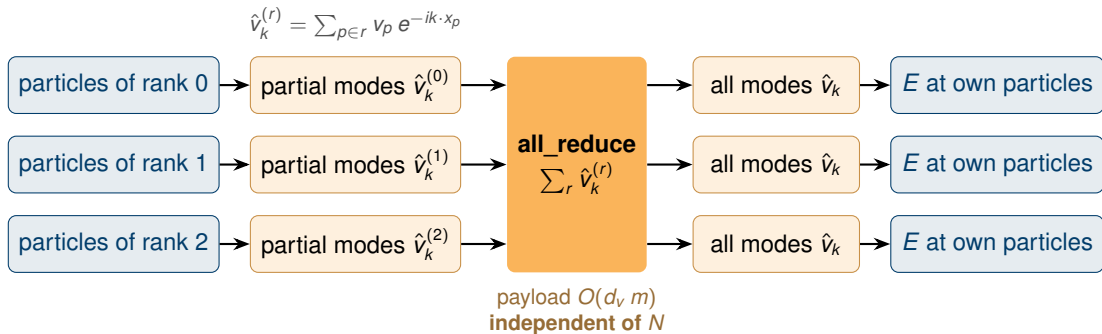
- Read volume per rank: $N \rightarrow N/t_p$
- Peak GPU memory **unchanged**: set downstream by activations / Fourier layers

Why more GPUs did not help



$$b \cdot n = \frac{BN}{W} \text{ independent of } t_p$$

PARALLEL INFERENCE



- Particle decomposition in picND: each GPU holds N/W particles
- Integrated quantities (e.g. field energy) also via all_reduce

WHAT IS IPPL?

- Independent **P**arallel **P**article **L**ayer: C++ library for **particle–mesh simulations** (e.g. PIC)
- Performance portable: **Kokkos** (CPU and GPU) + **MPI**
- Domain decomposition: each rank owns a subdomain and its particles
- Mini-apps such as Landau damping; PIF schemes implemented too

FIGURE: IPPL logo

drop the file in figures/
figures/ippl_logo.pdf|png

mini-app, e.g. Landau damping

IPPL: particles, fields, solvers

Kokkos

MPI

CPU · GPU

COUPLING TO IPPL

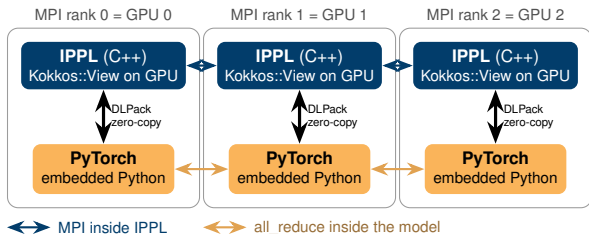
FIGURE: Solver or surrogate

IPPL-interop deck p.9: kick – drift – field
evaluation – kick; the field evaluation is
the numeric solver or the neural operator

`figures/ippl_solver_vs_surrogate.pdf|png`

- Time step unchanged: only the **field evaluation** is swapped for the surrogate
- Opt-in, header-only (pybind11 + DLPack), **no changes to IPPL**
- Python model loaded by name at runtime

COUPLING TO IPPL



- Time step unchanged: only the **field evaluation** is swapped for the surrogate
- Opt-in, header-only (pybind11 + DLPack), **no changes to IPPL**
- Python model loaded by name at runtime
- Same MPI rank hosts IPPL and the model: **rank i talks to rank i**
- Data stays on the GPU; Python writes E **directly into IPPL's memory**

SCALING RESULTS

NUFFT model, run **through the IPPL interop layer**: 400 time steps, 1–512 GPUs, up to 1.6 B particles

FIGURE: Strong + weak scaling

IPPL-interop deck p.13

[figures/ippl_inference_scaling.pdf](#)|png

FIGURE: Scaling efficiency

IPPL-interop deck p.14

[figures/ippl_scaling_efficiency.pdf](#)|png

efficiency: strong $E_s = P_0 T(P_0) / (P T(P))$, weak $E_w = T_{\text{base}} / T(P)$



Deep dive

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

TWO WAYS TO SPLIT THE PARTICLES

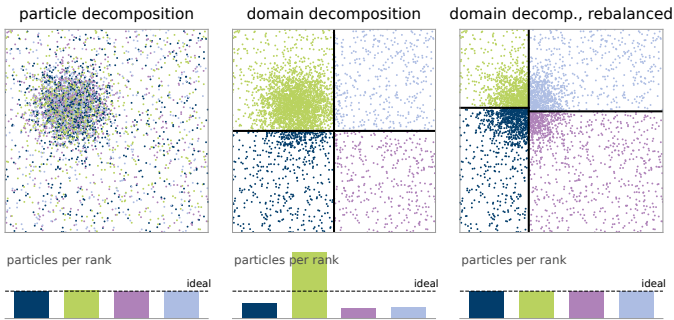


Illustration with synthetic particles (beam on a background), 4 ranks

Particle decomposition (our model, picND)

each rank: random subset over the **whole** domain; always balanced; Fourier modes duplicated

Domain decomposition (IPPL)

each rank: **one subdomain**; balance needs re-partitioning
→ subdomain boundaries move

THE BUG: LOCAL VS. GLOBAL BOUNDS

```
# valid() ✓  
lo = all_reduce(x.min(), MIN)  
hi = all_reduce(x.max(), MAX)  
  
# __call__() ✗  
lo, hi = x.min(), x.max()
```

schematic, not the actual code

- Particle decomp.: local $\approx$ global bounds $\rightarrow$ **harmless**, invisible in picND
- Domain decomp.: each rank stretches its subdomain to $[0, 2\pi)$ $\rightarrow$ ranks sum modes in **incompatible coordinates**
- Also: normalisation divides by max instead of $(\max - \min)$

✓ fixed early on — bounds are now all-reduced

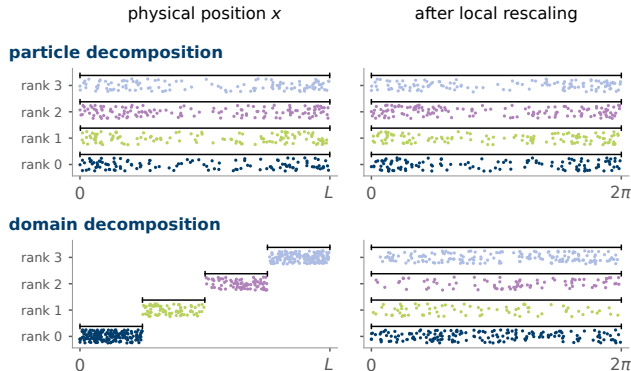


Illustration: synthetic 1D density $1 + 0.6 \cos x$, 4 ranks, $|k| \leq 4$

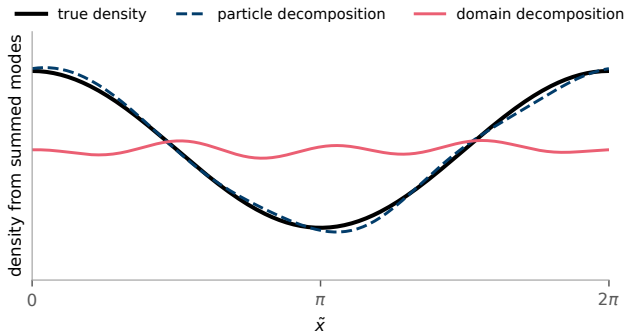
THE BUG: LOCAL VS. GLOBAL BOUNDS

```
# valid() ✓  
lo = all_reduce(x.min(), MIN)  
hi = all_reduce(x.max(), MAX)  
  
# __call__() ✗  
lo, hi = x.min(), x.max()
```

schematic, not the actual code

- Particle decomp.: local $\approx$ global bounds $\rightarrow$ **harmless**, invisible in picND
- Domain decomp.: each rank stretches its subdomain to $[0, 2\pi)$ $\rightarrow$ ranks sum modes in **incompatible coordinates**
- Also: normalisation divides by max instead of (max - min)

✓ fixed early on — bounds are now all-reduced



Density rebuilt from the all-reduced modes

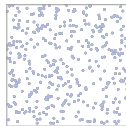
Illustration: synthetic 1D density $1 + 0.6 \cos x$, 4 ranks, $|k| \leq 4$

THE RISING NOISE FLOOR

```
# initial particles, every rank ✗  
cp.random.seed(152)  
  
# fix ✓  
cp.random.seed(152 + 100*rank)
```

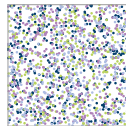
- Every GPU draws its N/W particles from the **same seed** → very similar particles on all ranks
- Effectively only $\approx N/W$ independent particles → noise floor **grows with the GPU count**
- One seed per rank → the noise floor no longer depends on W

seed(152) on every rank



400 distinct particles

seed(152 + 100*rank)



1600 distinct particles

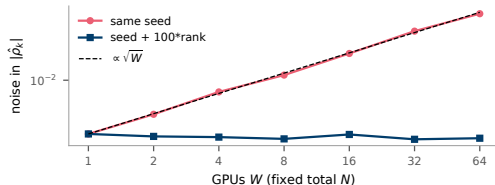


Illustration: numpy, uniform particles, fixed $N = 2^{16}$, modes $k = 1..8$, 20 trials

THE RISING NOISE FLOOR

```
# initial particles, every rank ❌
```

```
cp.random.seed(152)
```

```
# fix ✅
```

```
cp.random.seed(152 + 100*rank)
```

- Every GPU draws its N/W particles from the **same seed** → very similar particles on all ranks
- Effectively only $\approx N/W$ independent particles → noise floor **grows with the GPU count**
- One seed per rank → the noise floor no longer depends on W

FIGURE: Real run: before vs. after

your plot: seed(152) vs. seed(152 + 100*rank)

figures/noise_floor_before_after.pdf|png

ZERO-COPY: KOKKOS VIEW $\rightarrow$ PYTORCH VIA DLPACK

ParticleAttrib<Vector<double,3>

Kokkos::View<Vector<double,3>*>

shape (N, 3) of double

Kokkos::View

x_0

y_0

z_0

x_1

y_1

z_1

x_2

y_2

z_2

$\dots$

one GPU allocation, no copy

ZERO-COPY: KOKKOS VIEW → PYTORCH VIA DLPACK

ParticleAttrib<Vector<double,3>

Kokkos::View<Vector<double,3>*>

shape (N, 3) of double

Kokkos::View

x_0

y_0

z_0

x_1

y_1

z_1

x_2

y_2

z_2

...

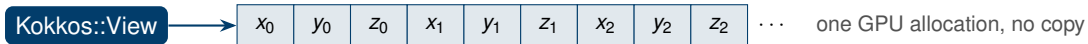
one GPU allocation, no copy

DLTensor{ data, device=CUDA,
dtype=float64, shape=(N,3),
strides=(3,1) } + deleter

Zero-copy only if standard layout, no padding:

`sizeof(Vector<T,D>) == D*sizeof(T)`
otherwise: **repack on the device** (still no host copy)

ZERO-COPY: KOKKOS VIEW → PYTORCH VIA DLPACK



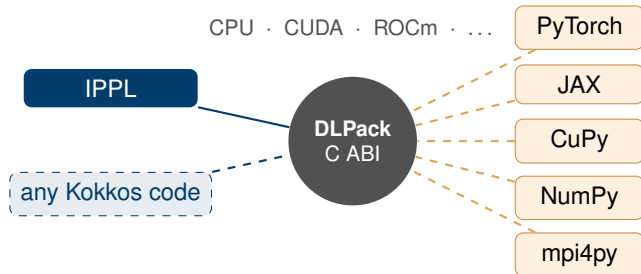
DLTensor{ data, device=CUDA,
dtype=float64, shape=(N,3),
strides=(3,1) } + deleter

Zero-copy only if standard layout, no padding:
`sizeof(Vector<T,D>) == D*sizeof(T)`
otherwise: **repack on the device** (still no host copy)

Lifetime

the capsule keeps the Kokkos View alive until PyTorch releases it

WHY DLPACK MATTERS BEYOND THIS PROJECT



- Open standard: NumPy, CuPy, PyTorch, TensorFlow, JAX, MXNet, TVM, Paddle, mpi4py
- Cross-hardware, plain C ABI → any language
- Our export side is generic over Kokkos Views; model side not tied to PyTorch



Status

TOWARDS A FOUNDATIONAL MODEL FOR PARTICLE-BASED METHODS

STATUS AND OUTLOOK

Works

- Sharded HDF5 reads in training
- Sharded inference in picND
- One RNG seed per rank: noise floor fixed
- Global bounds: bug fixed
- IPPL $\leftrightarrow$ PyTorch interop layer (zero-copy, E written in place)
- Inference scaling through IPPL: up to 512 GPUs, 1.6 B particles

In progress

- ADIOS2 / openPMD streaming (loose coupling)

Blocked

- Large jobs on JUWELS Booster (software updates)

Next

[TODO: your next steps]

ACKNOWLEDGEMENTS

- **Dr. Sriramkrishnan Muralikrishnan** (JSC) – supervisor
- **Chelsea John** – the foundation model is her PhD thesis; my work contributes to it
- NEOPIC is funded by **Helmholtz AI**
INF funding code ZT-I-PF-5-242

FIGURE: Logos
Helmholtz AI, JSC
(check logo usage rules)

Thank you!

Questions?